

CI/CD Pipeline - E-commerce Cart System

Table of Contents

- [Overview](#)
- [Pipeline Architecture](#)
- [Development Workflow](#)
- [Environment Strategy](#)
- [Pipeline Stages](#)
- [Quality Gates](#)
- [Deployment Strategy](#)
- [Monitoring and Observability](#)

Overview

This document outlines the Continuous Integration and Continuous Delivery (CI/CD) pipeline for the E-commerce Cart System. The pipeline is designed to ensure code quality, automate testing, and enable reliable deployments across multiple environments.

Pipeline Goals

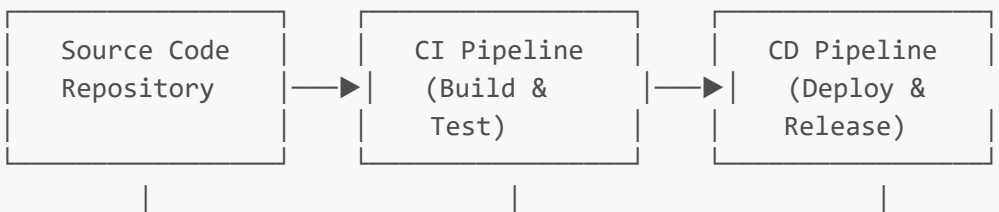
- Automated Quality Assurance:** Ensure code quality through automated testing and analysis
- Fast Feedback:** Provide quick feedback to developers on code changes
- Reliable Deployments:** Minimize deployment risks through automated processes
- Environment Consistency:** Maintain consistency across development, staging, and production
- Rollback Capability:** Enable quick rollbacks in case of issues

Pipeline Principles

- Everything as Code:** Infrastructure, configuration, and deployment scripts are version-controlled
- Automation First:** Manual interventions are minimized
- Fail Fast:** Issues are detected early in the pipeline
- Security by Design:** Security checks are integrated throughout the pipeline
- Observability:** Comprehensive logging and monitoring at every stage

Pipeline Architecture

High-Level Architecture



▼

Feature
Branches

▼

Artifact
Repository

▼

Environment
Management

Technology Stack

- **Version Control:** Git (GitHub/GitLab)
- **CI/CD Platform:** GitHub Actions / GitLab CI / Jenkins
- **Container Registry:** Docker Hub / AWS ECR / Google Container Registry
- **Infrastructure:** Kubernetes / Docker Swarm / AWS ECS
- **Monitoring:** Prometheus, Grafana, ELK Stack
- **Security:** SonarQube, OWASP ZAP, Trivy

Development Workflow

Git Flow Strategy

```
main (production)
├── develop (integration)
├── feature/user-authentication
├── feature/product-catalog
├── feature/shopping-cart
├── bugfix/payment-validation
└── hotfix/security-patch
```

Branch Naming Convention

```
feature/US-123-add-product-search
bugfix/ORDER-456-fix-payment-validation
hotfix/SEC-789-fix-sql-injection
release/v1.2.0
```

Commit Message Format

```
<type>(<scope>): <description>

[optional body]

[optional footer]
```

Types:

- **feat**: New feature
- **fix**: Bug fix
- **docs**: Documentation changes
- **style**: Code style changes
- **refactor**: Code refactoring
- **test**: Adding or updating tests
- **chore**: Maintenance tasks

Examples:

`feat(product): add product search functionality`

- Implement product search by name and description
- Add search filters by category and price range
- Include search result pagination

Closes #123

`fix(order): resolve payment validation issue`

- Fix credit card validation logic
- Add proper error handling for invalid cards
- Update payment error messages

Fixes #456

Environment Strategy

Environment Tiers

1. Development Environment

- **Purpose**: Local development and testing
- **Access**: Developers only
- **Data**: Mock data or small test datasets
- **Deployment**: Manual or automated on push to develop branch
- **URL**: `dev.ecommerce-cart.com`

2. Integration Environment

- **Purpose**: Integration testing and feature validation
- **Access**: Development team and QA
- **Data**: Anonymized production data
- **Deployment**: Automated on merge to develop branch
- **URL**: `staging.ecommerce-cart.com`

3. Pre-Production Environment

- **Purpose:** Final testing before production release
- **Access:** QA team and stakeholders
- **Data:** Production-like data
- **Deployment:** Automated on release branch creation
- **URL:** `preprod.ecommerce-cart.com`

4. Production Environment

- **Purpose:** Live application serving real users
- **Access:** End users and operations team
- **Data:** Real production data
- **Deployment:** Automated with approval gates
- **URL:** `ecommerce-cart.com`

Environment Configuration

```
# Environment-specific configuration
environments:
  development:
    database_url: postgresql://localhost:5432/ecommerce_dev
    redis_url: redis://localhost:6379
    log_level: debug
    features:
      payment_processing: false
      email_notifications: false

  staging:
    database_url: postgresql://staging-db:5432/ecommerce_staging
    redis_url: redis://staging-redis:6379
    log_level: info
    features:
      payment_processing: true
      email_notifications: false

  production:
    database_url: postgresql://prod-db:5432/ecommerce_prod
    redis_url: redis://prod-redis:6379
    log_level: warn
    features:
      payment_processing: true
      email_notifications: true
```

Pipeline Stages

1. Source Stage

Purpose: Trigger pipeline and prepare source code

Activities:

- Code checkout from repository
- Branch validation
- Commit message validation
- Trigger pipeline based on branch patterns

Configuration:

```
# GitHub Actions workflow trigger
on:
  push:
    branches: [ main, develop, feature/*, bugfix/*, hotfix/* ]
  pull_request:
    branches: [ main, develop ]
  release:
    types: [ published ]
```

2. Build Stage

Purpose: Compile and package the application

Activities:

- Install dependencies
- Run code linting
- Compile TypeScript
- Build Docker images
- Run unit tests
- Generate build artifacts

Configuration:

```
build:
  stage: build
  script:
    - npm ci
    - npm run lint
    - npm run build
    - npm run test:unit
    - docker build -t ecommerce-cart:$CI_COMMIT_SHA .
  artifacts:
    paths:
      - dist/
      - coverage/
    expire_in: 1 week
```

3. Test Stage

Purpose: Comprehensive testing of the application

Activities:

- Integration tests
- API tests
- End-to-end tests
- Performance tests
- Security scans

Configuration:

```
test:
  stage: test
  services:
    - postgres:14
    - redis:7
  script:
    - npm run test:integration
    - npm run test:e2e
    - npm run test:performance
    - npm run security:scan
  coverage: '/All files[^|]*\|[^|]*s+([\d\.]+)/'
```

4. Quality Gate Stage

Purpose: Ensure code quality and security standards

Activities:

- Code coverage analysis
- Static code analysis
- Security vulnerability scanning
- Dependency vulnerability scanning
- Code review automation

Configuration:

```
quality-gate:
  stage: quality
  script:
    - npm run sonar:analysis
    - npm run security:audit
    - npm run dependency:check
  rules:
    - coverage >= 80%
    - security_issues = 0
    - critical_vulnerabilities = 0
```

5. Package Stage

Purpose: Create deployment artifacts

Activities:

- Tag Docker images
- Push to container registry
- Create deployment manifests
- Generate release notes

Configuration:

```
package:
  stage: package
  script:
    - docker tag ecommerce-cart:$CI_COMMIT_SHA $REGISTRY/ecommerce-
      cart:$CI_COMMIT_SHA
    - docker tag ecommerce-cart:$CI_COMMIT_SHA $REGISTRY/ecommerce-cart:latest
    - docker push $REGISTRY/ecommerce-cart:$CI_COMMIT_SHA
    - docker push $REGISTRY/ecommerce-cart:latest
    - generate-release-notes
  only:
    - main
    - develop
```

6. Deploy Stage

Purpose: Deploy to target environments

Activities:

- Environment-specific configuration
- Database migrations
- Application deployment
- Health checks
- Smoke tests

Configuration:

```
deploy-staging:
  stage: deploy
  environment:
    name: staging
    url: https://staging.ecommerce-cart.com
  script:
    - kubectl apply -f k8s/staging/
    - kubectl rollout status deployment/ecommerce-cart
```

```
- npm run smoke:test
only:
  - develop

deploy-production:
  stage: deploy
  environment:
    name: production
    url: https://ecommerce-cart.com
  script:
    - kubectl apply -f k8s/production/
    - kubectl rollout status deployment/ecommerce-cart
    - npm run smoke:test
  when: manual
  only:
    - main
```

☒ Quality Gates

1. Code Quality Gates

Code Coverage

- **Minimum Coverage:** 80%
- **Critical Paths:** 100%
- **New Code:** 90%

```
coverage-gate:
  script:
    - npm run test:coverage
    - |
      COVERAGE=$(cat coverage/coverage-summary.json | jq -r '.total.lines.pct')
      if (( $(echo "$COVERAGE < 80" | bc -l) )); then
        echo "Code coverage ($COVERAGE%) is below minimum threshold (80%)"
        exit 1
      fi
```

Code Quality Metrics

- **Cyclomatic Complexity:** < 10 per function
- **Lines of Code:** < 50 per function
- **Code Duplication:** < 5%
- **Technical Debt:** < 5%

```
quality-metrics:
  script:
    - npm run sonar:analysis
```



```

- |
  COMPLEXITY=$(curl -s "$SONAR_URL/api/measures/component?
componentKey=$PROJECT_KEY&metricKeys=complexity" | jq -r
'.component.measures[0].value')
  if [ "$COMPLEXITY" -gt 10 ]; then
    echo "Cyclomatic complexity ($COMPLEXITY) exceeds threshold (10)"
    exit 1
  fi

```

2. Security Gates

Vulnerability Scanning

- **Critical Vulnerabilities:** 0
- **High Vulnerabilities:** 0
- **Medium Vulnerabilities:** < 5

```

security-scan:
  script:
    - npm audit --audit-level=moderate
    - trivy image $REGISTRY/ecommerce-cart:$CI_COMMIT_SHA
    - zap-baseline.py -t https://staging.ecommerce-cart.com

```

Dependency Security

- **Outdated Dependencies:** < 10%
- **Known Vulnerabilities:** 0
- **License Compliance:** All dependencies compliant

```

dependency-check:
  script:
    - npm audit
    - npm outdated
    - license-checker --summary

```

3. Performance Gates

Performance Benchmarks

- **API Response Time:** < 200ms (95th percentile)
- **Database Query Time:** < 100ms (95th percentile)
- **Memory Usage:** < 512MB per container
- **CPU Usage:** < 70% under load

```
performance-test:
  script:
    - npm run test:performance
    - |
      RESPONSE_TIME=$(jq -r '.p95' performance-results.json)
      if (( $(echo "$RESPONSE_TIME > 200" | bc -l) )); then
        echo "API response time ($RESPONSE_TIME ms) exceeds threshold (200 ms)"
        exit 1
      fi
```

4. Functional Gates

Automated Testing

- **Unit Tests:** 100% pass
- **Integration Tests:** 100% pass
- **End-to-End Tests:** 100% pass
- **Smoke Tests:** 100% pass

```
functional-tests:
  script:
    - npm run test:unit
    - npm run test:integration
    - npm run test:e2e
    - npm run smoke:test
```

Deployment Strategy

1. Blue-Green Deployment

Strategy Overview

- **Blue Environment:** Current production version
- **Green Environment:** New version being deployed
- **Switch:** Traffic is switched from blue to green
- **Rollback:** Quick rollback by switching back to blue

Implementation

```
blue-green-deploy:
  script:
    # Deploy to green environment
    - kubectl apply -f k8s/green/
    - kubectl rollout status deployment/ecommerce-cart-green
```

```

# Run health checks
- kubectl exec deployment/ecommerce-cart-green -- curl -f
http://localhost:3000/health

# Switch traffic
- kubectl patch service ecommerce-cart -p '{"spec":{"selector":
{"version":"green"}}}'

# Verify green is healthy
- sleep 30
- curl -f https://ecommerce-cart.com/health

# Scale down blue
- kubectl scale deployment ecommerce-cart-blue --replicas=0

```

2. Canary Deployment

Strategy Overview

- **Canary:** Small percentage of traffic to new version
- **Gradual Rollout:** Increase traffic percentage over time
- **Monitoring:** Monitor metrics and rollback if issues detected
- **Full Rollout:** 100% traffic to new version

Implementation

```

canary-deploy:
  script:
    # Deploy canary
    - kubectl apply -f k8s/canary/
    - kubectl rollout status deployment/ecommerce-cart-canary

    # Route 10% traffic to canary
    - kubectl patch service ecommerce-cart -p '{"spec":{"traffic":
[{"weight":90,"version":"stable"}, {"weight":10,"version":"canary"}]}}'

    # Monitor for 5 minutes
    - sleep 300

    # Check metrics
    - |
      ERROR_RATE=$(curl -s $PROMETHEUS_URL/api/v1/query?
query=rate(http_requests_total{status=~"5.."}[5m]))
      if (( $(echo "$ERROR_RATE > 0.05" | bc -l) )); then
        echo "Error rate ($ERROR_RATE) exceeds threshold (5%)"
        kubectl patch service ecommerce-cart -p '{"spec":{"traffic":
[{"weight":100,"version":"stable"}]}}'
        exit 1
      fi

```

```
# Increase to 50%
- kubectl patch service ecommerce-cart -p '{"spec":{"traffic":
[{"weight":50,"version":"stable"}, {"weight":50,"version":"canary"}]}}'

# Monitor for 10 minutes
- sleep 600

# Full rollout
- kubectl patch service ecommerce-cart -p '{"spec":{"traffic":
[{"weight":100,"version":"canary"}]}}'
```

3. Rolling Update Deployment

Strategy Overview

- **Gradual Replacement:** Replace pods one by one
- **Health Checks:** Verify each pod before proceeding
- **Rollback:** Automatic rollback on failure
- **Zero Downtime:** Continuous service availability

Implementation

```
rolling-update:
  script:
    - kubectl set image deployment/ecommerce-cart ecommerce-
      cart=$REGISTRY/ecommerce-cart:$CI_COMMIT_SHA
    - kubectl rollout status deployment/ecommerce-cart --timeout=300s

# Verify deployment
- kubectl get pods -l app=ecommerce-cart
- curl -f https://ecommerce-cart.com/health
```

Monitoring and Observability

1. Application Monitoring

Health Checks

```
// Health check endpoint
app.get('/health', async (req, res) => {
  try {
    // Database health check
    await db.query('SELECT 1');

    // Redis health check
    await redis.ping();
```

```

// External service health checks
const paymentServiceHealth = await checkPaymentService();
const emailServiceHealth = await checkEmailService();

const health = {
  status: 'healthy',
  timestamp: new Date().toISOString(),
  version: process.env.APP_VERSION,
  checks: {
    database: 'healthy',
    redis: 'healthy',
    paymentService: paymentServiceHealth,
    emailService: emailServiceHealth
  }
};

res.status(200).json(health);
} catch (error) {
  res.status(503).json({
    status: 'unhealthy',
    timestamp: new Date().toISOString(),
    error: error.message
  });
}
});

```

Metrics Collection

```

import prometheus from 'prom-client';

// Metrics
const httpRequestDuration = new prometheus.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code']
});

const httpRequestTotal = new prometheus.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status_code']
});

// Middleware
app.use((req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = (Date.now() - start) / 1000;
    httpRequestDuration
      .labels(req.method, req.route?.path || req.path, res.statusCode.toString())

```

```

        .observe(duration);

        httpRequestTotal
        .labels(req.method, req.route?.path || req.path, res.statusCode.toString())
        .inc();
    });

    next();
  });

// Metrics endpoint
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', prometheus.register.contentType);
  res.end(await prometheus.register.metrics());
});

```

2. Infrastructure Monitoring

Kubernetes Monitoring

```

# Prometheus configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s

    scrape_configs:
      - job_name: 'kubernetes-pods'
        kubernetes_sd_configs:
          - role: pod
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
            action: keep
            regex: true
          - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
            action: replace
            target_label: __metrics_path__
            regex: (.+)
          - source_labels: [__address__,
            __meta_kubernetes_pod_annotation_prometheus_io_port]
            action: replace
            regex: ([^:]+)(?::\d+)?(\d+)?
            replacement: $1:$2
            target_label: __address__

```

Alerting Rules

```
# Alerting configuration
groups:
- name: ecommerce-cart
  rules:
    - alert: HighErrorRate
      expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "High error rate detected"
        description: "Error rate is {{ $value }} errors per second"

    - alert: HighResponseTime
      expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m])) > 0.2
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "High response time detected"
        description: "95th percentile response time is {{ $value }} seconds"

    - alert: PodDown
      expr: up{job="ecommerce-cart"} == 0
      for: 1m
      labels:
        severity: critical
      annotations:
        summary: "Pod is down"
        description: "Pod {{ $labels.pod }} is down"
```

3. Logging Strategy

Structured Logging

```
import winston from 'winston';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'ecommerce-cart' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

```
});

// Request logging middleware
app.use((req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info('HTTP Request', {
      method: req.method,
      url: req.url,
      statusCode: res.statusCode,
      duration,
      userAgent: req.get('User-Agent'),
      ip: req.ip
    });
  });

  next();
});
```

Log Aggregation

```
# Fluentd configuration for log aggregation
apiVersion: v1
kind: ConfigMap
metadata:
  name: fluentd-config
data:
  fluent.conf: |
    <source>
      @type tail
      path /var/log/containers/*.log
      pos_file /var/log/fluentd-containers.log.pos
      tag kubernetes.*
      read_from_head true
      <parse>
        @type json
        time_format %Y-%m-%dT%H:%M:%S.%NZ
      </parse>
    </source>

    <match kubernetes.**>
      @type elasticsearch
      host elasticsearch
      port 9200
      logstash_format true
      logstash_prefix k8s
      <buffer>
        @type file
        path /var/log/fluentd-buffers/kubernetes.system.buffer
```



```
flush_mode interval
retry_type exponential_backoff
flush_interval 5s
retry_forever false
retry_max_interval 30
chunk_limit_size 2M
queue_limit_length 8
overflow_action block
</buffer>
</match>
```

Pipeline Configuration Examples

GitHub Actions Workflow

```
name: CI/CD Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

env:
  REGISTRY: ghcr.io/${{ github.repository_owner }}
  IMAGE_NAME: ecommerce-cart

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:14
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
```

```
steps:
- uses: actions/checkout@v3

- name: Setup Node.js
  uses: actions/setup-node@v3
  with:
    node-version: '18'
    cache: 'npm'

- name: Install dependencies
  run: npm ci

- name: Run linting
  run: npm run lint

- name: Run tests
  run: npm run test:coverage
  env:
    DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test
    REDIS_URL: redis://localhost:6379

- name: Upload coverage
  uses: codecov/codecov-action@v3
```

```
security:
  runs-on: ubuntu-latest
  needs: test
```

```
steps:
- uses: actions/checkout@v3

- name: Run security audit
  run: npm audit --audit-level=moderate

- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ env.github.sha }
    format: 'sarif'
    output: 'trivy-results.sarif'

- name: Upload Trivy scan results
  uses: github/codeql-action/upload-sarif@v2
  with:
    sarif_file: 'trivy-results.sarif'
```

```
build:
  runs-on: ubuntu-latest
  needs: [test, security]
  if: github.event_name == 'push'
```

```
steps:
- uses: actions/checkout@v3
```

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v2
- name: Log in to Container Registry
uses: docker/login-action@v2
with:
 - registry: \${{ env.REGISTRY }}
 - username: \${{ github.actor }}
 - password: \${{ secrets.GITHUB_TOKEN }}
- name: Build and push Docker image
uses: docker/build-push-action@v4
with:
 - context: .
 - push: true
 - tags: |
 - \${{ env.REGISTRY }}/\${{ env.IMAGE_NAME }}:\${{ github.sha }}
 - \${{ env.REGISTRY }}/\${{ env.IMAGE_NAME }}:latest
 - cache-from: type=gha
 - cache-to: type=gha,mode=max

deploy-staging:

```
runs-on: ubuntu-latest
needs: build
if: github.ref == 'refs/heads/develop'
environment: staging
```

steps:

- uses: actions/checkout@v3
- name: Deploy to staging
run: |
 - echo "Deploying to staging environment..."
 - # Add deployment commands here
- name: Run smoke tests
run: |
 - echo "Running smoke tests..."
 - # Add smoke test commands here

deploy-production:

```
runs-on: ubuntu-latest
needs: build
if: github.ref == 'refs/heads/main'
environment: production
```

steps:

- uses: actions/checkout@v3
- name: Deploy to production
run: |
 - echo "Deploying to production environment..."
 - # Add deployment commands here

```
- name: Run smoke tests
  run: |
    echo "Running smoke tests..."
    # Add smoke test commands here
```

GitLab CI Pipeline

```
stages:
  - test
  - security
  - build
  - deploy

variables:
  DOCKER_DRIVER: overlay2
  DOCKER_TLS_CERTDIR: "/certs"

test:
  stage: test
  services:
    - postgres:14
    - redis:7
  variables:
    POSTGRES_DB: test
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
  script:
    - npm ci
    - npm run lint
    - npm run test:coverage
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
    paths:
      - coverage/
    expire_in: 1 week

security:
  stage: security
  script:
    - npm audit --audit-level=moderate
    - trivy image $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  allow_failure: true

build:
  stage: build
  image: docker:20.10.16
  services:
```

```

- docker:20.10.16-dind
script:
- docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
- docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
- |
  if [ "$CI_COMMIT_BRANCH" = "$CI_DEFAULT_BRANCH" ]; then
    docker tag $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA $CI_REGISTRY_IMAGE:latest
    docker push $CI_REGISTRY_IMAGE:latest
  fi
only:
- main
- develop

deploy-staging:
stage: deploy
image: alpine:latest
before_script:
- apk add --no-cache curl
script:
- echo "Deploying to staging..."
- curl -X POST $STAGING_WEBHOOK_URL
environment:
name: staging
url: https://staging.ecommerce-cart.com
only:
- develop

deploy-production:
stage: deploy
image: alpine:latest
before_script:
- apk add --no-cache curl
script:
- echo "Deploying to production..."
- curl -X POST $PRODUCTION_WEBHOOK_URL
environment:
name: production
url: https://ecommerce-cart.com
when: manual
only:
- main

```

This CI/CD pipeline documentation provides a comprehensive framework for implementing automated, reliable, and secure deployment processes for the E-commerce Cart System.